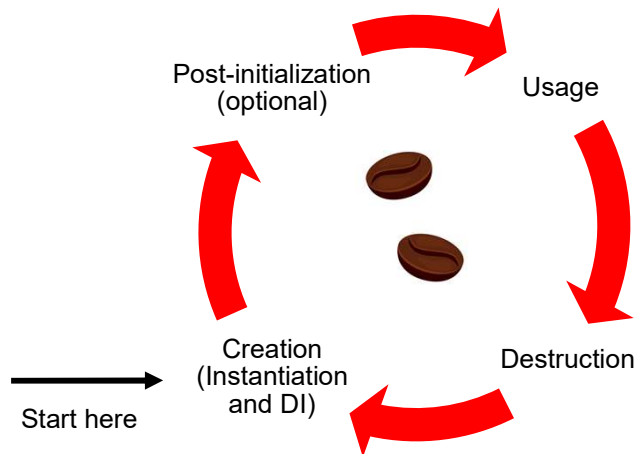


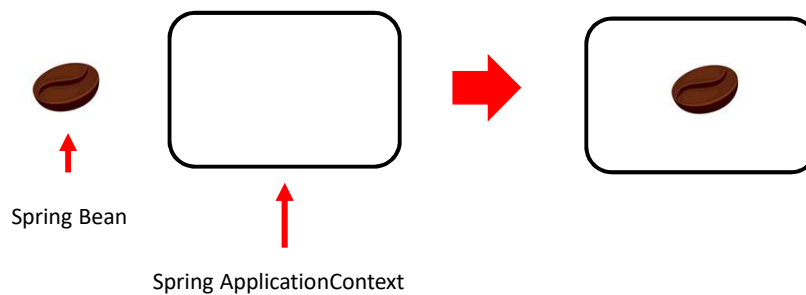
Spring Bean lifecycle in a nutshell



32

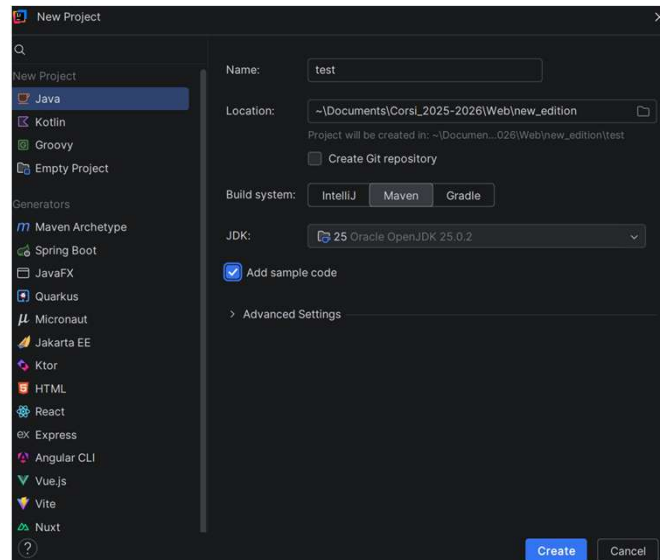
Step 1: Creation (Instantiation)

- What we want to do:
 - Creating an ApplicationContext and a Bean
 - Adding the Bean to the ApplicationContext so that it can «see» it



33

Creating a Maven project



34

Adding dependencies in the pom.xml

```
<dependencies>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>7.0.3</version>
  </dependency>
</dependencies>
```

35

Step 1: Creation (Instantiation)

- There are 2 (3) ways to achieve our goal:

1) Using the @Bean annotation and a Java Configuration class

How?

The Bean is created by a method of the Configuration class.
Such a method is annotated using @Bean

Step by step (esempio1, esempio1bis):

- Create a Maven project and add in the pom.xml dependencies for Spring context
- Write a POJO for your Bean
- Write a Configuration class having a method called as your Bean and annotate it using @Bean
- Write a main class for your app in which you create the ApplicationContext

36

Step 1: Creation (Instantiation)

- There are 2 (3) ways to achieve our goal:

1) Using the @Bean annotation and a Java Configuration class

When is it used?

When the class whose object we want to create is not defined in the app (e.g. basic Java datatypes as String or Integer, objects defined in third party libraries) or when we want to have a fine-grained control over the initial values of the object properties

37

Step 1: Creation (Instantiation)

- There are 2 (3) ways to achieve our goal:

2) Using stereotype annotations

How?

The Bean is created as an instance of a class annotated using `@Component` (or its derivatives as `@Controller`, `@Repository...`)

Step by step (esempio2):

- Create a Maven project and add in the pom.xml dependencies for Spring context
- Write a POJO for your Bean and annotate it using `@Component`
- Write an empty Configuration class annotated with `@ComponentScan` to indicate where the Bean class is
- Write a main class for your app in which you create the `ApplicationContext`

38

Step 1: Creation (Instantiation)

- There are 2 (3) ways to achieve our goal:

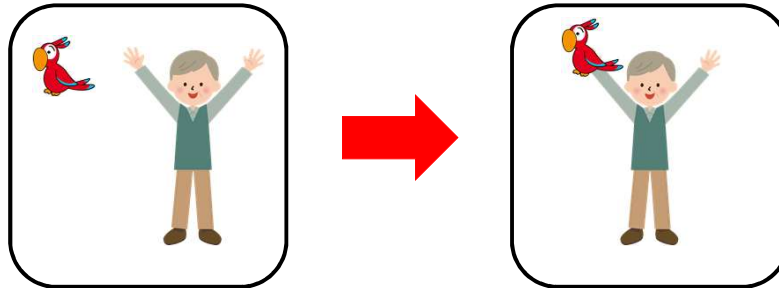
2) Using stereotype annotations

When is it used?

When you use only classes defined in the app for which you do not need to have a fine-grained control

39

Step 1: Creation (Dependencies Injection)



- Let suppose now to have two Beans: a parrot and a person
- We want to establish a relationship between them so that one Bean can delegate to the other in the implementation of their responsibilities

40

Step 1: Creation (Dependencies Injection)

- As previously mentioned, there are 3 ways to do it. We will focus on the **Constructor injection method** only
- **With @Bean and a Configuration class:** in the Configuration class the Person method takes a parrot as a parameter that then it is passed to the Person's Constructor.
- **With stereotype annotations:** the Person's Constructor is called directly by Spring that passes also a parrot as a parameter. The @Autowired annotation is used.

41

Step 2: Post initialization

- This is an optional step to complete the initialization of beans (e.g. setting property of the bean) when stereotype annotations are used

How?

Writing an init method in your bean class, and annotating it using `@PostConstruct`.

Add in the pom.xml the dependency for using such an annotation

```
<dependency>
  <groupId>jakarta.annotation</groupId>
  <artifactId>jakarta.annotation-api</artifactId>
  <version>2.1.1</version>
</dependency>
```

When is it used?

It is used with the stereotype annotations only

42 **Step by step (esempio3)**

Step 3 and Step 4

- Step 3: After creation and possible post-inizialitation, the bean is part of the Spring context and ready to be used by the other components within the application context
- Step 4: When the application context is closed (`.close()`), the bean will go through the destruction phase. This phase is handled by the Spring container to perform cleanup or release resources held by the bean.

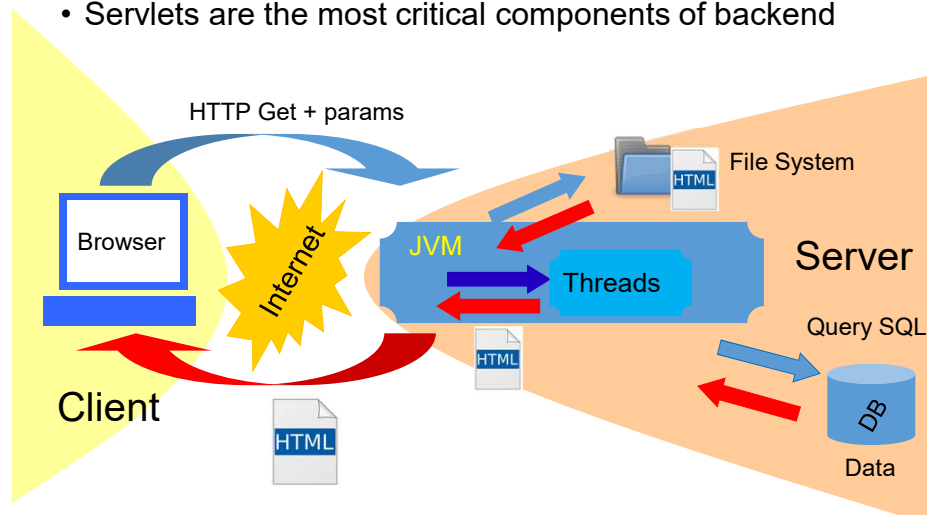
43

Ready to use Spring for developing web apps!

44

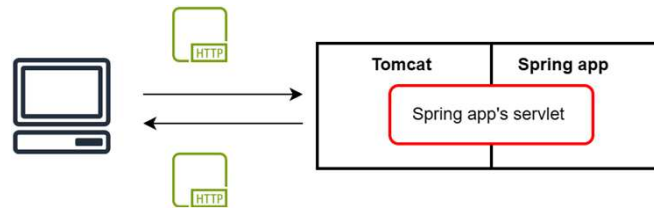
One step back

- Servlets are the most critical components of backend



In a Spring web app, a servlet is the entry point to our app's logic

A Spring «raw» architecture for web app



- A servlet Container take care of what is needed to communicate with the client using HTTP. It transforms requests and responses into objects the app understands
- You do not need to write servlets anymore, Spring uses just an already-written servlet as entry point of the app 's logic
- The already-written servlet is registered so that it is called for any path of the client's request

46

Programming web apps is complex!

- On the server side we have:
 - Many components
 - Many interactions (clients, DB, and so on)
 - Many functionalities
- Are there suitable ways to organize a web app structure and behavior on the server?

YES! Using design patterns

47

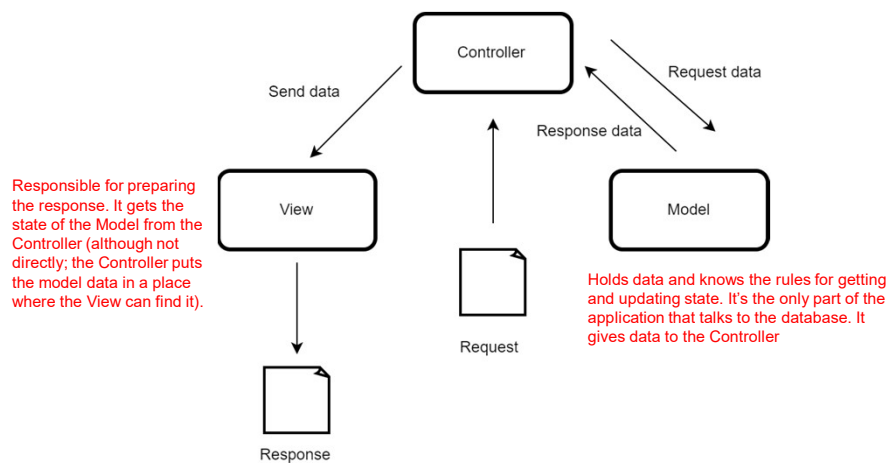
Let's have a look!

- Design patterns are reusable solutions to common problems in software design
- For web applications, many of the most important patterns belong to the architectural design patterns family. These patterns define how the main components of an application are organized and how they interact
- The Model-View-Controller (MVC) is an architectural design pattern separating the code into **three** components:
 - Model
 - View
 - Controller

48

MVC

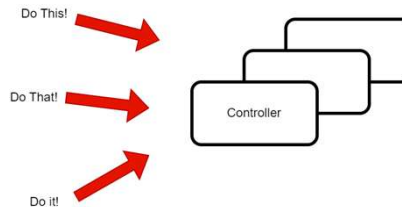
It is the entry point of the requests: it receives requests, it invokes the Model to obtain and modify data, it chooses the View



49

Great! But...

- In classic MVC, each request is managed by a different Controller...

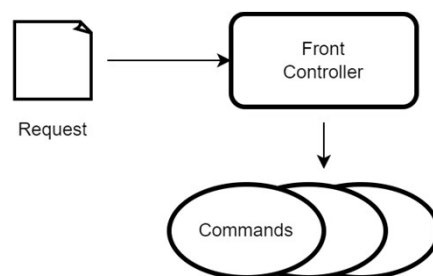


- Having many controllers is not a good idea: they do always the same stuff (e.g. logging, authentication, error management)!
- What we can do?

50

Front Controller

- The rationale: every request first passes through a single central entry point: the Front Controller



- The Front Controller: receives requests, apply common operations, send the request to the right Controller

51

What about Spring?

- In Spring, the Front Controller pattern and the MVC pattern work closely together
- Spring, under the hood, provides an infrastructure based on the Front Controller pattern, while developers work at a higher level of abstraction using the MVC pattern to clearly separate business logic, presentation, and control
- Let's have a look!

52